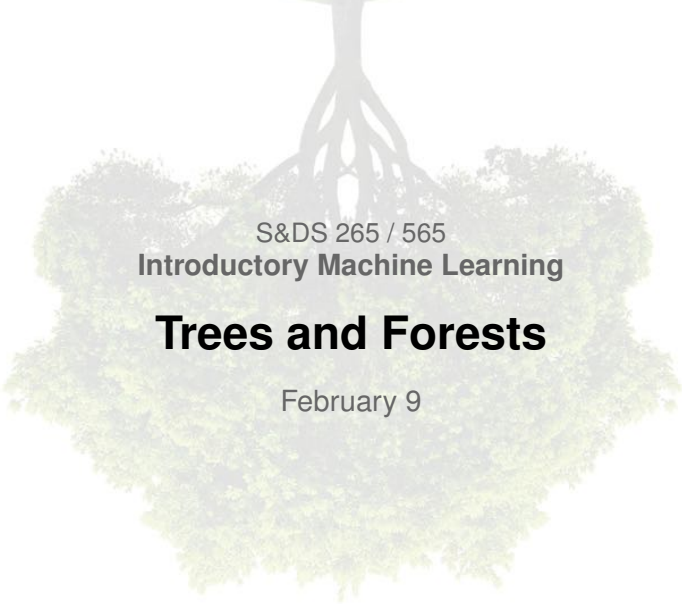




S&DS 265 / 565
Introductory Machine Learning

Trees and Forests

February 9

Yale

Reminders

- Assn 2 out; due Feb 19
- Quiz 2 this week!
- Midterm in class on Wednesday, March 4
- Join us in OH

Trees in context

- Trees are a “classic” machine learning topic
- Not often used today
- But still very important conceptually
- Trees and forests will expand our understanding of the bias-variance tradeoff

Classification and Regression Trees (CART)

Trees provide ways of modeling nonlinear relationships by carving out *rectangular regions* in the feature space.

Classification and Regression Trees (CART)

Trees provide ways of modeling nonlinear relationships by carving out *rectangular regions* in the feature space.

- Response variables can be categorical or quantitative

Classification and Regression Trees (CART)

Trees provide ways of modeling nonlinear relationships by carving out *rectangular regions* in the feature space.

- Response variables can be categorical or quantitative
- Yields a set of **interpretable decision rules**

Classification and Regression Trees (CART)

Trees provide ways of modeling nonlinear relationships by carving out *rectangular regions* in the feature space.

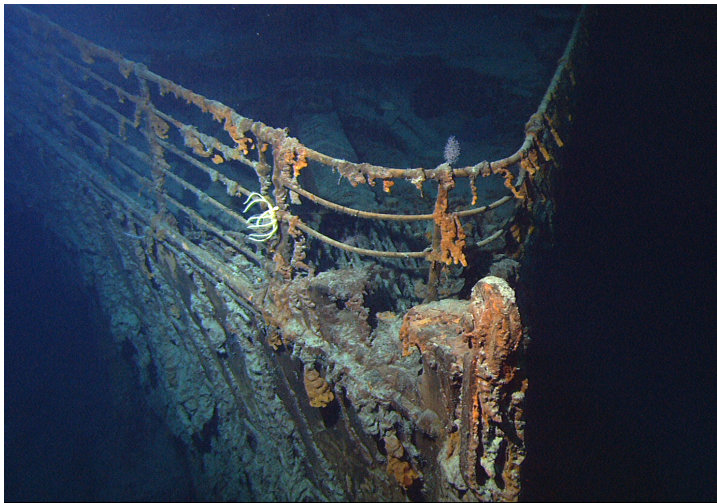
- Response variables can be categorical or quantitative
- Yields a set of **interpretable decision rules**
- Predictive ability is mediocre, *but* can be improved by combining multiple trees (resampling, ensemble methods)

Titanic in culture



[https://en.wikipedia.org/wiki/Titanic_\(1997_film\)](https://en.wikipedia.org/wiki/Titanic_(1997_film))

Titanic in culture



https://en.wikipedia.org/wiki/Wreck_of_the_Titanic

Titanic data

Sign In

Getting Started Prediction Competition

Titanic: Machine Learning from Disaster

Start here! Predict survival on the Titanic and get familiar with ML basics

 Kaggle · 17,691 teams · Ongoing

Overview

Data

Notebooks

Discussion

Leaderboard

Rules

Join Competition

Overview

Description

Evaluation

Frequently Asked Questions

👋🎉 **Ahoy, welcome to Kaggle! You're in the right place.**

This is the legendary Titanic ML competition – the best, first challenge for you to dive into ML competitions and familiarize yourself with how the Kaggle platform works.

The competition is simple: use machine learning to create a model that predicts which passengers survived the Titanic shipwreck.

Read on or watch the video below to explore more details. Once you're ready to start competing, click on the ["Join Competition button"](#) to create an account and gain access to the [competition data](#). Then check out [Alexis Cook's Titanic Tutorial](#) that walks you through step by step how to make your first submission!

Titanic data

- **Survived:** Outcome of survival (0 = No; 1 = Yes)
- **Pclass:** Socio-economic class (1 = Upper class; 2 = Middle class; 3 = Lower class)
- **Name:** Name of passenger
- **Sex:** Sex of the passenger
- **Age:** Age of the passenger (Some entries contain NaN)
- **SibSp:** Number of siblings and spouses of the passenger aboard
- **Parch:** Number of parents and children of the passenger aboard
- **Ticket:** Ticket number of the passenger
- **Fare:** Fare paid by the passenger
- **Cabin** Cabin number of the passenger (Some entries contain NaN)
- **Embarked:** Port of embarkation of the passenger (C = Cherbourg; Q = Queenstown; S = Southampton)

Trees



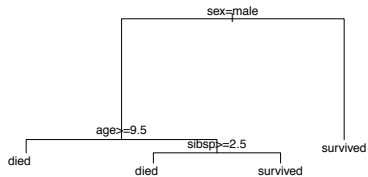
Trees



Trees

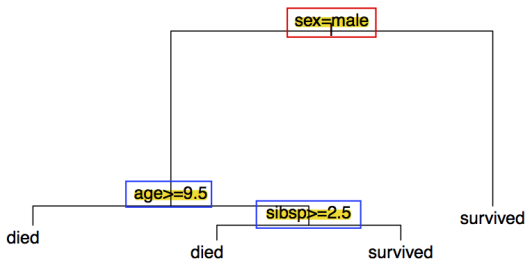


Modeling Titanic survival:



Tree terminology

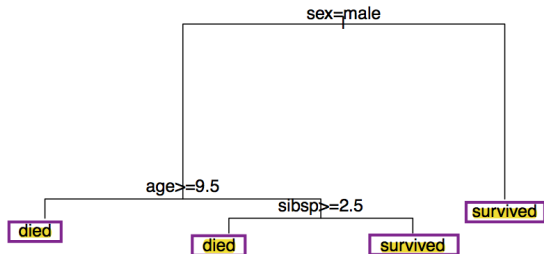
Internal nodes are points where the predictor space is split.



The internal node at the top is the **root** of the tree.

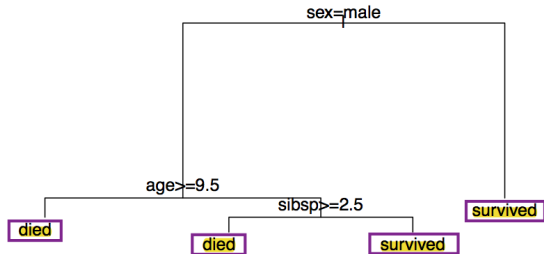
Tree terminology

Terminal nodes (or **leaves**) are the ends of the tree where no further splitting occurs.



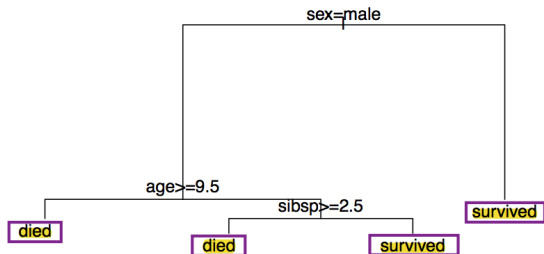
Tree terminology

Terminal nodes (or **leaves**) are the ends of the tree where no further splitting occurs.



Denote these J regions as $R_1, \dots, R_J$.

Tree terminology



- $R_1 = \{i : \text{sex}_i = \text{male} \cap \text{age}_i \geq 9.5\}$
- $R_2 = \{i : \text{sex}_i = \text{male} \cap \text{age}_i < 9.5 \cap \text{sibsp}_i \geq 2.5\}$
- $R_3 = \{i : \text{sex}_i = \text{male} \cap \text{age}_i < 9.5 \cap \text{sibsp}_i < 2.5\}$
- $R_4 = \{i : \text{sex}_i \neq \text{male}\}$

Let's go to the Titanic demo

Bias-variance

- Nodes are split by greedily choosing the best question (greatest reduction in error)
- As tree is grown deeper, bias decreases
- But the variance increases
- How to choose the right size of tree?

Stopping criterion

Once we stop, we relabel the terminal nodes to be $R_1, \dots, R_J$ and compute $\bar{y}_{R_j}$ (means within each region) to serve as $\hat{y}$ values.

But when do we stop?

Stopping criterion

Once we stop, we relabel the terminal nodes to be $R_1, \dots, R_J$ and compute $\bar{y}_{R_j}$ (means within each region) to serve as $\hat{y}$ values.

But when do we stop?

Some possibilities:

- number of observations in a node has reached a minimum

Stopping criterion

Once we stop, we relabel the terminal nodes to be $R_1, \dots, R_J$ and compute $\bar{y}_{R_j}$ (means within each region) to serve as $\hat{y}$ values.

But when do we stop?

Some possibilities:

- number of observations in a node has reached a minimum
- depth of tree has reached a maximum

Stopping criterion

Once we stop, we relabel the terminal nodes to be $R_1, \dots, R_J$ and compute $\bar{y}_{R_j}$ (means within each region) to serve as $\hat{y}$ values.

But when do we stop?

Some possibilities:

- number of observations in a node has reached a minimum
- depth of tree has reached a maximum
- grow until no further splits can reduce RSS by some amount

Stopping criterion

Once we stop, we relabel the terminal nodes to be $R_1, \dots, R_J$ and compute $\bar{y}_{R_j}$ (means within each region) to serve as $\hat{y}$ values.

But when do we stop?

Some possibilities:

- number of observations in a node has reached a minimum
- depth of tree has reached a maximum
- grow until no further splits can reduce RSS by some amount

Stopping criterion

Once we stop, we relabel the terminal nodes to be $R_1, \dots, R_J$ and compute $\bar{y}_{R_j}$ (means within each region) to serve as $\hat{y}$ values.

But when do we stop?

Some possibilities:

- number of observations in a node has reached a minimum
- depth of tree has reached a maximum
- grow until no further splits can reduce RSS by some amount

Many options – resulting in tuning parameters that are hard to deal with.

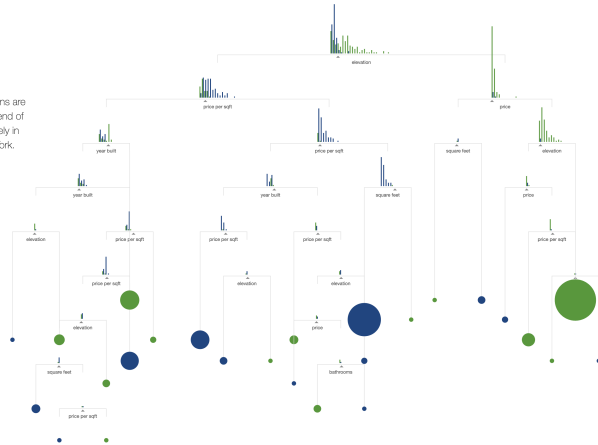
Tree pruning

Another way to get around the overfitting problem is to grow a large tree and then **prune** it back.

Typically, pruning involves looking at subtrees of the fully-grown tree, and comparing how well the subtrees perform.

Tree pruning

You could even continue to add branches until the tree's predictions are **100% accurate**, so that at the end of every branch, the homes are purely in San Francisco or purely in New York.



Tree pruning

How do we prune?

- cross validation
- cost-complexity pruning

Cost-complexity pruning

$$\begin{aligned}\text{Minimize:} \quad & \text{Loss}(T) + \lambda \{\# \text{ of nodes in } T\} \\ & = \sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \lambda |T|\end{aligned}$$

λ is a tuning parameter that controls for the complexity of the model.

Cost-complexity pruning

$$\begin{aligned}\text{Minimize: } & \text{Loss}(T) + \lambda \{\# \text{ of nodes in } T\} \\ & = \sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \lambda |T|\end{aligned}$$

λ is a tuning parameter that controls for the complexity of the model.

- $\lambda = 0$ implies the full tree

Cost-complexity pruning

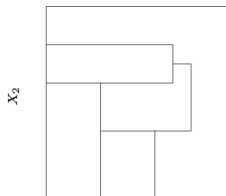
$$\begin{aligned}\text{Minimize:} \quad & \text{Loss}(T) + \lambda \{\# \text{ of nodes in } T\} \\ & = \sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \lambda |T|\end{aligned}$$

λ is a tuning parameter that controls for the complexity of the model.

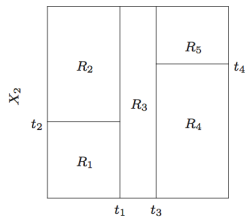
- $\lambda = 0$ implies the full tree
- Larger λ implies higher penalty for complexity of model

Tree pruning

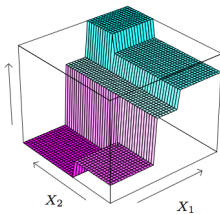
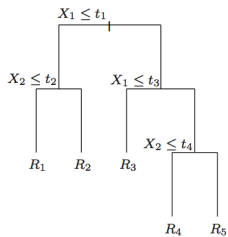
- 1 Grow a big tree on a *training set*.
- 2 Obtain a nested set of subtrees $T_L \subset \dots \subset T_2 \subset T_1 \subset T_0$ corresponding to a sequence of λ values.
- 3 Use (leave-one-out or k -fold) cross-validation to identify the subtree/ λ that does best.



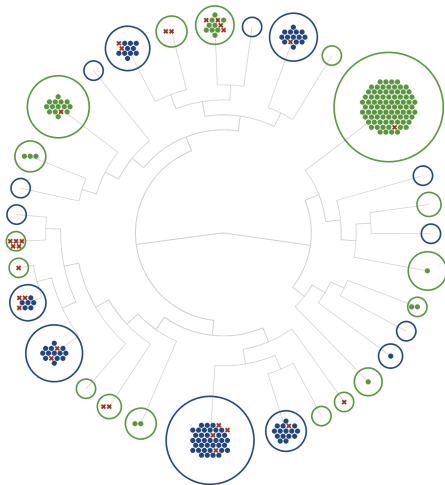
X_1



X_1



Beautiful demo <http://www.r2d3.us/>



This demo gives a nice description of bias-variance tradeoff for trees. It's well worth a look!

Trees vs. other methods

Decision trees are similar in spirit to k -nearest neighbors.

Trees vs. other methods

Decision trees are similar in spirit to k -nearest neighbors.

- Both produce simple predictions (averages/maximally occurring) based on “neighborhoods” in the predictor space.

Trees vs. other methods

Decision trees are similar in spirit to k -nearest neighbors.

- Both produce simple predictions (averages/maximally occurring) based on “neighborhoods” in the predictor space.
- Neighborhoods chosen very differently

Trees vs. other methods

Recall that linear regression fits models of the form

$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j$$

Trees vs. other methods

Recall that linear regression fits models of the form

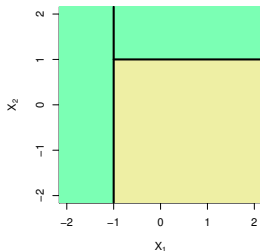
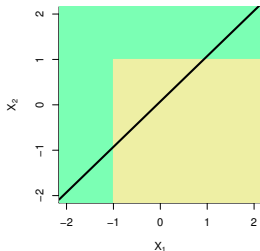
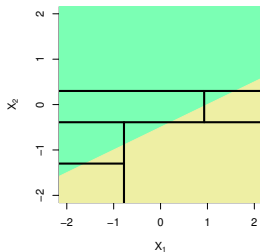
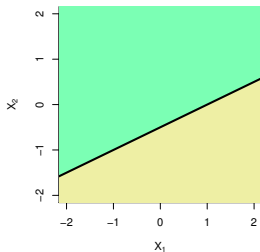
$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j$$

Regression trees are like fitting linear regression models with a bunch of indicators

$$f(X) = \sum_{j=1}^J \beta_j \mathbb{1} \{X \in R_j\}$$

Trees vs. other methods

Are trees always better than linear methods?



Summary so far

- Trees give interpretable, nonlinear prediction rules
- Deep trees have low bias, high variance
- Shallow trees have high bias, low variance
- Deep trees are pruned back using cross-validation to find best bias/variance tradeoff.